

Submission PY1-PS2-SUB02

Question PY1-PS2-Q01

work / setup:

1. range(5) gives 0 through 4
2. keep even i: 0, 2, 4

crossed-out arithmetic: [smudged]

FINAL (maybe): [0, 4, 16] plus 1

Question PY1-PS2-Q02

work / setup:

1. counts = Counter(xs)
2. for x in xs: if counts[x] == 1: return x

crossed-out arithmetic: [smudged]

FINAL (maybe): scan counts, then scan in order; return first count==1 else None; $O(n)$ time, $O(n)$ space plus 1

Question PY1-PS2-Q03

work / setup:

1. A single pass fixes only adjacent inversions encountered once.

crossed-out arithmetic: [smudged]

FINAL (maybe): repeat passes or track the shrinking unsorted suffix; one pass alone is not a full sort plus 1

Question PY1-PS2-Q04

work / setup:

1. linear scan: up to 128 comparisons, $\Theta(n)$

crossed-out arithmetic: [smudged]

FINAL (maybe): linear $\Theta(n)$, up to 128; binary $\Theta(\log n)$, about 8 plus 1

Question PY1-PS2-Q05

work / setup:

1. Removing shifts later elements left while the iterator advances.

crossed-out arithmetic: [smudged]

FINAL (maybe): mutation shifts elements and skips checks; use a filtered copy or iterate over a copy plus 1

Question PY1-PS2-Q06

work / setup:

1. range(7) gives 0 through 6
2. keep even i: 0, 2, 4, 6

crossed-out arithmetic: [smudged]

FINAL (maybe): [0, 4, 16, 36] plus 1

Question PY1-PS2-Q07

work / setup:

1. counts = Counter(xs)

2. for x in xs: if counts[x] == 1: return x

crossed-out arithmetic: [smudged]

FINAL (maybe): scan counts, then scan in order; return first count==1 else None; $O(n)$ time, $O(n)$ space plus 1

Question PY1-PS2-Q08

work / setup:

1. A single pass fixes only adjacent inversions encountered once.

crossed-out arithmetic: [smudged]

FINAL (maybe): repeat passes or track the shrinking unsorted suffix; one pass alone is not a full sort plus 1